

Reviewer Pack — Minimal Rank of Toric Varieties and DPLL Search Tree Depth

Entry f97a65b6f2af · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 22:25:42 UTC

Minimal Rank of Toric Varieties and DPLL Search Tree Depth

Entry ID: f97a65b6f2af **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 22:25:42 UTC

1. Conjecture

Field A (mathematical branch): Toric Geometry **Field B** (complexity object): DPLL Search Tree Complexity

Statement:

[‘For every satisfiable Boolean formula F with n variables, the minimal rank of its associated toric variety is asymptotically equal to the depth of the DPLL search tree for F .’, ‘ $E[\text{MinimalRank}(V(F))] = \Theta(\text{Depth}(S(F)))$ ’, ‘where $V(F)$ denotes the toric variety associated with F and $S(F)$ is the DPLL search tree for F .’]

Rationale (proposer’s reasoning):

[‘Toric geometry provides a geometric representation of Boolean formulas, which could potentially reveal hidden structures in the satisfiability problem.’, ‘By establishing a connection between the algebraic properties of toric varieties and the complexity of solving satisfiability problems, this conjecture might expose new insights into computational hardness.’]

Taxonomy category: ToricGeometryDPLL (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): adc4c944d716f793

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the mean depth of the DPLL search tree and the minimal rank of the toric variety are within a factor of 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -minimal rank toric varieties AND DPLL search tree depth -toric geometry AND DPLL complexity of search trees -asymptotic relationship between toric variety rank and DPLL tree depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for DPLL algorithm
def dpll(formula, assignment, literals):
    if not formula:
        return True
    literal = next(lit for lit in literals if lit not in assignment and -lit not in assignment)
    positive = literal > 0
    new_assignment = assignment.copy()
    new_assignment[literal] = positive

    # Simplify the formula by substituting the literal
    simplified_formula = []
    for clause in formula:
        if literal in clause:
            continue
        if -literal in clause:
            clause.remove(-literal)
        if not clause:
            return False
    simplified_formula.append(clause)

    if dpll(simplified_formula, new_assignment, literals):
        return True

# Backtrack
del new_assignment[literal]
new_assignment[-literal] = not positive
simplified_formula = []
for clause in formula:
    if -literal in clause:
```

```

        continue
    if literal in clause:
        clause.remove(literal)
        if not clause:
            return False
    simplified_formula.append(clause)

    return dpll(simplified_formula, new_assignment, literals)

def dpll_search_tree_depth(formula, n):
    return 1 + max(dpll_search_tree_depth([c for c in formula if literal in c], n) for literal in formula)

# Constructive mapping from Boolean formula to toric variety
def construct_toric_variety(formula):
    # This is a placeholder function. Replace with actual implementation.
    return 0

# Main trial function
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    formula = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if all(lit not in clause and -lit not in clause for lit in formula):
            formula.append(clause)

    toric_variety_rank = construct_toric_variety(formula)
    dpll_depth = dpll_search_tree_depth(formula, n)

    return {
        "metric_name": "Rank of Toric Variety vs DPLL Depth",
        "metric_value": abs(toric_variety_rank - dpll_depth),
        "instances_tested": 1,
        "conjecture_holds": abs(toric_variety_rank - dpll_depth) <= max(toric_variety_rank, dpll_depth),
        "counterexample": "" if toric_variety_rank == dpll_depth else f"Rank: {toric_variety_rank}"
    }

# Main execution
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")

    mean_metric = sum(r['metric_value'] for r in results) / len(results)
    std_metric = math.sqrt(sum((r['metric_value'] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")

```

```

else:
    first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39e653b2.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39e653b2.py", line 71, in run_trial
    if all(lit not in clause and -lit not in clause for lit in formula):
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_39e653b2.py", line 71, in <genexpr>
    if all(lit not in clause and -lit not in clause for lit in formula):
        ^^^^
TypeError: bad operand type for unary -: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be verified. | next: Re-run the test with proper error handling to verify the conjecture's support condition.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13801
2	preregistration	ollama_remote	glm4:latest	0	0	9610
3	novelty	ollama_remote	glm4:latest	0	0	8645
4	novelty	ollama_remote	glm4:latest	0	0	9820
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11300
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11015
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11034
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11508
9	judge	ollama_remote	glm4:latest	0	0	11979

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98713 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f97a65b6f2af.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f97a65b6f2af.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f97a65b6f2af.tar.gz (if generated)